Relatório e análise crítica sobre a comparação de complexidade de tempo de métodos de ordenação

Matheus dos Santos Moreira
Mestrado Profissional em Computação Aplicada - Algoritmos e Programação
Prof. Dr. Valéria Farinazzo Martins
06 de março de 2026

1. Introdução

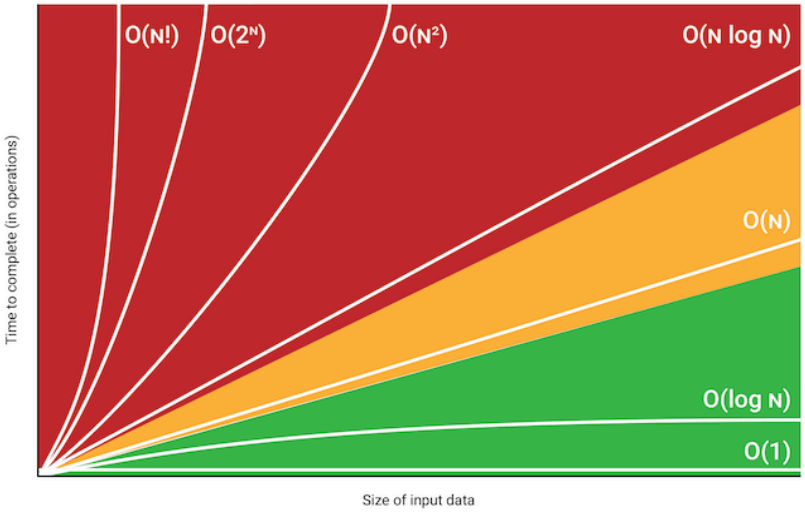
Este relatório é um projeto auxiliar da disciplina de algoritmos e programação. Nele, pretendemos introduzir o conceito de análise assintótica para análise de complexidade de tempo, demonstrar cinco algoritmos de ordenação e relacionar o tempo de execução desses algoritmos com o que se espera a partir da análise assintótica com vetores de diferentes tamanhos e diferentes ordenações iniciais. Fica fora do escopo do trabalho a análise de complexidade de espaço dos algoritmos.

2. Fundamentação teórica

2.1. Análise assintótica

Análise assintótica é uma forma de analisar a performance de algoritmos e descrever matematicamente como o tempo de execução cresce conforme o tamanho da entrada (CORMEN et al., 2022). Essas duas grandezas são conhecidas, respectivamente, como complexidade de tempo e de espaço. As definições matemáticas são expressadas com uma ou mais variáveis e uma breve explicação, normalmente n, onde n é o tamanho do *input* inicial.

A seguinte imagem mostra as definições assintóticas mais comuns entre algoritmos conhecidos:



2.2. Bolha

O algoritmo bolha melhorado é um algoritmo simples de ordenação de itens. A complexidade de tempo deste algoritmo é $O(n^2)$. Ele é conhecido por bolha porque os itens maiores movem-se para o lado direito do vetor da mesma forma que bolhas sobem em um copo com líquido. É um algoritmo de implementação simples e extremamente conhecido. Porém, devido à sua complexidade quadrática, torna-se pouco eficiente para vetores grandes.

2.3. Seleção

O algoritmo de seleção é também um algoritmo simples de ordenação de itens com complexidade de tempo $O(n^2)$. Ele se baseia em percorrer o vetor procurando o menor elemento e colocá-lo na primeira posição disponível, repetindo esse processo até que todo o vetor esteja ordenado. Apesar de sua lógica simples e implementação direta, o algoritmo também apresenta desempenho limitado para grandes volumes de dados devido ao seu crescimento quadrático.

2.4. Inserção

O algoritmo de inserção, também conhecido como *insertion sort*, é bastante conhecido e de simples implementação, possuindo complexidade de tempo média de $O(n^2)$. Ele funciona de forma semelhante à maneira como cartas são organizadas em um jogo de baralho, inserindo cada novo elemento na posição correta dentro da parte já ordenada do vetor. Em situações onde o vetor já está parcialmente ordenado, o algoritmo pode apresentar desempenho melhor do que outros algoritmos quadráticos.

2.5. Mergesort

O Mergesort, diferente dos algoritmos anteriores, é um algoritmo de ordenação mais avançado que utiliza o método de dividir e conquistar para alcançar uma complexidade de tempo média de $O(n \log n)$. Ele funciona dividindo o vetor em partes menores até que cada subvetor possua apenas um elemento e então combina essas partes novamente de forma ordenada. Esse método garante um desempenho estável e eficiente mesmo para vetores grandes.

2.6. Quicksort

O algoritmo Quicksort é também um algoritmo de ordenação avançado que utiliza o método de dividir e conquistar. Ele é eficiente e possui complexidade de tempo média de $O(n \log n)$, sendo um dos algoritmos de ordenação mais utilizados na prática. Porém, o algoritmo pode apresentar complexidade de tempo de até $O(n^2)$ caso o pivô, que é o elemento utilizado como referência para a divisão do vetor, seja escolhido de forma inadequada. Mesmo assim, em muitos cenários práticos ele apresenta excelente desempenho.

3. Método

Os testes foram executados com a linguagem de programação *Rust* e o tempo de execução foi calculado em segundos com a biblioteca padrão de tempo da linguagem, *std::time::Instant*. Durante a execução, foram criados 9 vetores no total, ordenados de forma crescente, decrescente e aleatória, de 1.000, 10.000 e 100.000 itens. Com o intuito de eliminar outliers, cada algoritmo foi executado 5 vezes em cada vetor, totalizando 225 execuções. Para o cálculo do tempo final, o programa faz uma média com todos os tempos armazenados de cada combinação e exporta um arquivo .xlsx com 6 páginas, todas são agrupadas por padrão de inicialização, três são de resumo e três apresentam informações sobre todas as execuções para cada padrão.

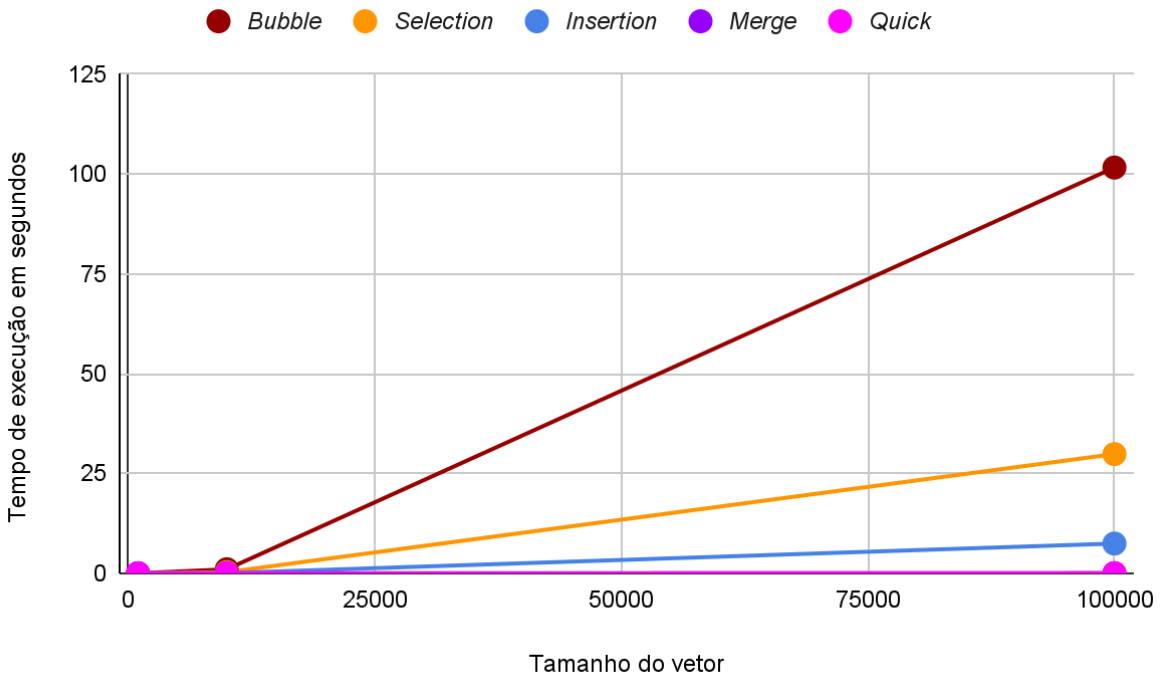
O projeto foi executado em uma CPU *Apple* M4 de 10 núcleos, 4 de performance e 6 de eficiência e 24GB de RAM. O código completo do projeto se encontra no GitHub:
https://github.com/mathmore0000/mestrado/tree/main/disciplinas/algoritmos%20e%20programacao/projetos/01%20-%20m%C3%A9todos%20de%20ordena%C3%A7%C3%A3o

4. Resultados

Os resultados foram consistentes e as execuções dentro dos grupos apontam para o mesmo padrão.

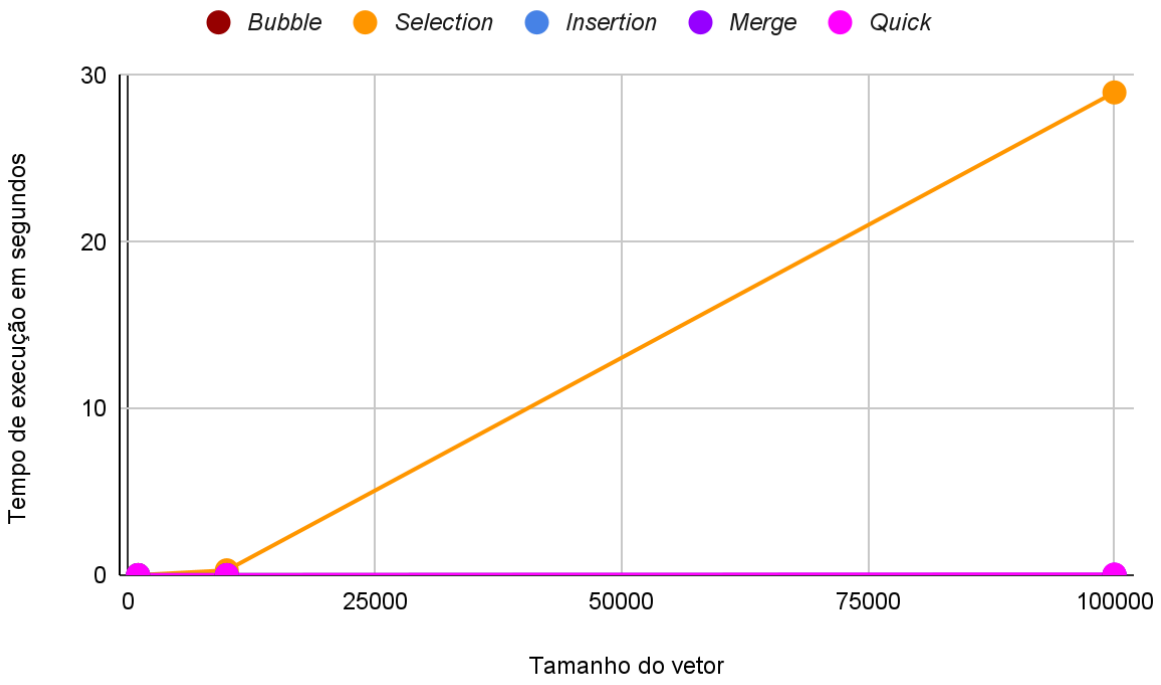
4.1. Vetor randômico

Para o grupo do vetor aleatório, o algoritmo de ordenação *quicksort* foi consistentemente o mais veloz, sendo seguido pelo *mergesort*, em terceiro lugar, o algoritmo de inserção e em quarto e quinto, respectivamente, seleção e o algoritmo bolha melhorado. Para uma visualização mais clara temos o seguinte gráfico do grupo de inicialização randomizado:

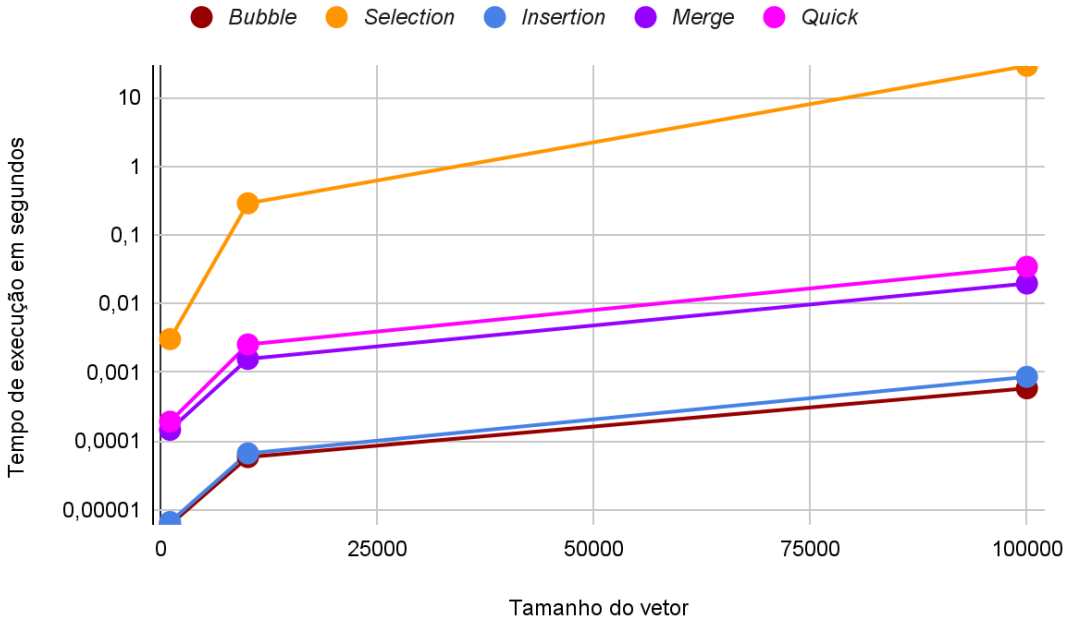


4.2. Vetor crescente

O grupo de inicialização em ordem crescente muda a ordem de eficiência padrão completamente. Nesse grupo a ordem da execução mais rápida para a mais lenta é bolha, inserção, merge, quick e por último seleção:

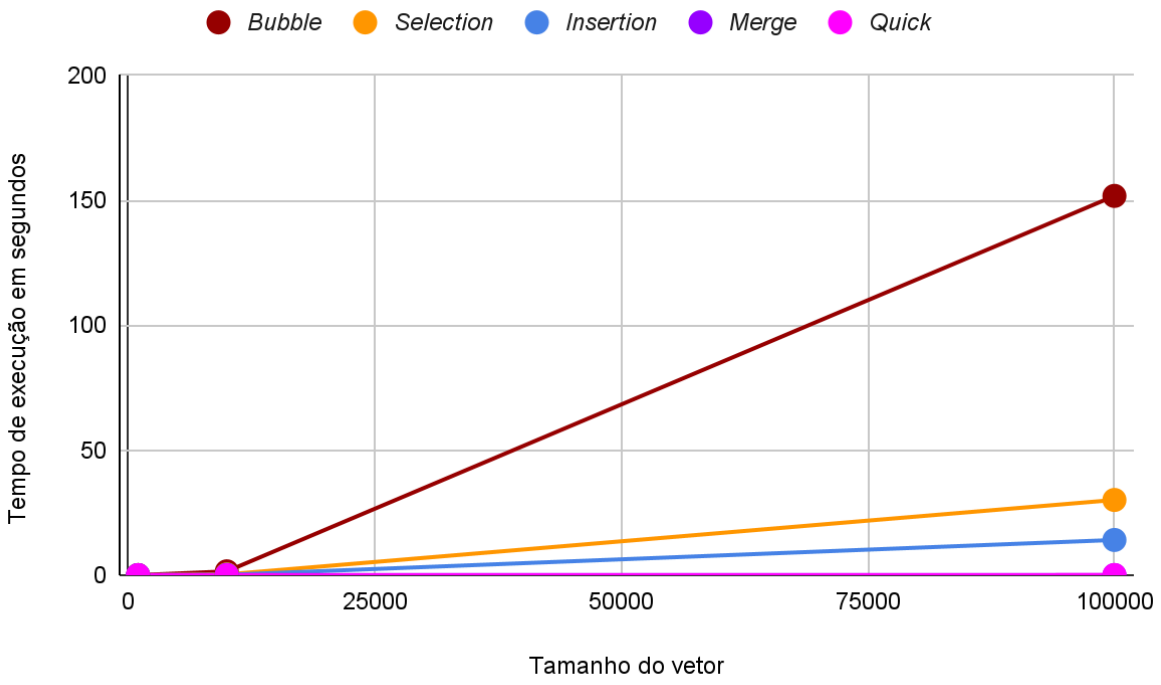


Apresento também o mesmo gráfico, com o eixo Y em escala logarítmica para facilitar a distinção entre os algoritmos mais velozes:



4.3. Vetor decrescente

Abaixo, o gráfico apresenta o tempo de execução de cada algoritmo quando o vetor é inicializado ordenado de forma decrescente:



5. Análise crítica

Nesta seção buscamos relacionar os resultados com a fundamentação teórica e explorar, junto com os resultados, possíveis interpretações equivocadas que podem surgir quando analisamos apenas o lado teórico da análise de desempenho.

5.1. Eficiência dos algoritmos

Conforme os resultados demonstram, a análise assintótica de complexidade de tempo fornece uma boa aproximação do comportamento dos algoritmos analisados. À medida que o tamanho do vetor aumenta, observa-se um crescimento significativo no tempo de execução dos algoritmos com complexidade quadrática, enquanto os algoritmos com complexidade O(n log n) mantêm tempos significativamente menores.

Embora os experimentos tenham sido realizados em um conjunto limitado de tamanhos de entrada, os resultados obtidos são consistentes com o comportamento esperado a partir da análise teórica.

5.2. Análise assintótica relacionada à interpretação dos resultados

É interessante observar que a análise assintótica descreve o comportamento dos algoritmos principalmente para valores muito grandes de entrada. Em cenários práticos, como os analisados neste relatório, fatores constantes que normalmente são ignorados pela notação assintótica ainda podem influenciar significativamente o tempo de execução observado. Uma possível explicação

para algumas diferenças observadas entre algoritmos com a mesma ordem assintótica está nas constantes presentes nas expressões de complexidade quando não utilizamos a simplificação da notação assintótica. Por exemplo, o número médio de comparações realizadas pelo bubble sort pode ser descrito aproximadamente por n(n-1)/2, enquanto no insertion sort o número médio de comparações é frequentemente representado por cerca de n(n-1)/4. Embora ambos possuam complexidade assintótica $O(n^2)$, essas diferenças constantes podem contribuir para que um algoritmo apresente melhor desempenho que outro em experimentos práticos com tamanhos de entrada limitados.

Esse tipo de diferença sugere que, embora a notação assintótica seja adequada para descrever o comportamento de crescimento dos algoritmos, ela não captura todos os fatores que influenciam o tempo real de execução. Elementos como constantes multiplicativas, características do input e detalhes de implementação também podem impactar o desempenho observado.

Outro aspecto relevante observado nos experimentos é que a ordenação inicial do vetor pode impactar significativamente o desempenho de determinados algoritmos. No caso do insertion sort e do bubble sort melhorado, vetores já ordenados reduzem drasticamente o número de operações necessárias, resultando em tempos de execução muito menores. Dessa forma, os resultados obtidos sugerem que a análise assintótica continua sendo uma ferramenta útil para compreender o crescimento do tempo de execução, mas que fatores práticos como constantes, padrões de entrada e implementação também desempenham papel relevante no desempenho observado.

6. Conclusão

Os experimentos realizados neste trabalho demonstram que os resultados observados estão, de maneira geral, alinhados com o comportamento esperado a partir da análise assintótica dos algoritmos de ordenação analisados. Algoritmos com complexidade quadrática apresentaram crescimento significativo no tempo de execução conforme o tamanho do vetor aumentou, enquanto algoritmos com complexidade O(n log n) mantiveram tempos de execução significativamente menores.

No entanto, os resultados também indicam que fatores práticos, como constantes multiplicativas, características do input e detalhes da implementação, podem influenciar o desempenho observado em cenários reais. Dessa forma, embora a análise assintótica seja uma ferramenta fundamental para compreender o comportamento de algoritmos de maneira genérica, ela não representa necessariamente todo o contexto em que um algoritmo é executado.

Nesse sentido, os experimentos apresentados neste relatório sugerem que a análise assintótica continua sendo uma ferramenta válida para compreender o crescimento do tempo de execução, mas que sua interpretação deve considerar que a notação simplificada oculta alguns fatores relevantes. Assim, a combinação entre análise teórica e testes experimentais se mostra uma abordagem mais completa para compreender o comportamento de algoritmos em aplicações reais.

7. Referências

CORMEN, Thomas H.; LEISERSON, Charles E.; RIVEST, Ronald L.; STEIN, Clifford.
Introduction to Algorithms. 4. ed. Cambridge: MIT Press, 2022.

MOREIRA, Matheus. Implementação dos algoritmos de ordenação utilizados neste trabalho.
Disponível em: https://github.com/mathmore0000/mestrado/tree/main/disciplinas/algoritmos%20e%20programacao/projetos/01%20-%20m%C3%A9todos%20de%20ordena%C3%A7%C3%A3o
Acesso em: 06 mar. 2026.